

Relazione progetto Gestione di Reti - Nicola Brillante

Credenziali

Nicola Brillante 675733 n.brillante@studenti.unipi.it

Obiettivo

Il programma è un **monitor di traffico di rete con classificazione applicativa dei protocolli**.

L'obiettivo è osservare il traffico che transita su un'interfaccia di rete e rispondere a due domande:

- **Quanto** traffico passa, misurato come throughput (byte al secondo) su una determinata scheda di rete del nostro PC?
- **Di che tipo** è quel traffico, cioè quale protocollo applicativo lo genera (HTTP, TLS, DNS, QUIC, ecc.)?

La seconda domanda è la parte più interessante di questo progetto, perchè l'identificazione del protocollo applicativo non si basa sul numero di porta ma su **Deep Packet Inspection** tramite la libreria nDPI, che analizza il contenuto effettivo dei pacchetti e calcola il loro protocollo applicativo.

Il programma distingue inoltre tre categorie di traffico rispetto all'host che esegue il monitor: traffico in **ingresso**, in **uscita**, e di **transito** (pacchetti che non coinvolgono l'host ma sono visibili grazie alla modalità promiscua della scheda di rete).

Per semplicità le scritte [IN] ,[OUT] e [TRANSIT] indicano se, all'interno di quella finestra di un secondo, il primo pacchetto osservato per quel protocollo è partito dalla mia interfaccia (OUT), è arrivato verso di essa (IN), oppure non la coinvolge (TRANSIT).

Descrizione

La strategia applicata si articola su quattro livelli:

1. **Cattura dei pacchetti.** Si usa **libpcap** per acquisire i pacchetti dall'interfaccia in modalità promiscua. Un filtro **BPF** compilato e installato nel kernel scarta a monte tutto ciò che non è un pacchetto IP, riducendo il carico sullo spazio utente.
2. **Ricostruzione dei flussi.** Il punto chiave è che la nDPI non può classificare un singolo pacchetto isolato: ha bisogno di osservare l'evoluzione di una conversazione (per esempio l'handshake TLS). Per questo il programma raggruppa i pacchetti in **flussi**, identificati dalla **5-tupla** (IP sorgente, IP destinazione, porta sorgente, porta destinazione, protocollo di trasporto). La chiave viene **normalizzata** ordinando i due IP, così che le due direzioni della

stessa conversazione ($A \rightarrow B$ e $B \rightarrow A$) confluiscono nello stesso flusso. I flussi sono memorizzati in una **tabella hash** per accesso in tempo costante.

3. **Classificazione con nDPI.** Per ogni flusso si mantiene una struttura di stato `ndpi_flow_struct` dove la libreria accumula le informazioni viste pacchetto dopo pacchetto. Si invoca `ndpi_detection_process_packet` finché nDPI non emette un verdetto; se dopo un numero limite di pacchetti il protocollo resta ignoto, si forza una stima con `ndpi_detection_giveup` (basata su porta e IP noti). Una volta identificato il flusso, lo si marca come "concluso" per non ripetere inutilmente l'analisi sui pacchetti successivi.
4. **Aggregazione e finestre temporali.** Le statistiche vengono accumulate per **protocollo** in una seconda tabella hash e riportate a **finestre di un secondo**. Al termine di ogni finestra i contatori vengono azzerati e si riparte; i flussi, invece, sopravvivono tra una finestra e l'altra, perché la loro identificazione può richiedere più di un secondo.
Infine il programma applica il **drop dei privilegi** (parte da root per aprire il socket di cattura, poi scende all'utente `nobody`) come misura di sicurezza, e contiene controlli difensivi sui pacchetti troncati o malformati per evitare letture fuori dai buffer.

Funzioni utilizzate

1. `get_local_ip(const char interface)`

Recupera l'indirizzo IPv4 associato all'interfaccia di rete e lo salva nella variabile globale `MIO_IP`. Viene chiamata **una sola volta all'avvio**, dentro il `main`, prima di iniziare la cattura. Il suo risultato serve poi a `update_ip` per distinguere il traffico locale (IN/OUT) da quello di transito. Riceve `const char *interface`, cioè il nome dell'interfaccia (es. "enp0s3"), che arriva dalla riga di comando nel caso l'utente voglia effettuare una cattura live. Non restituisce nulla: scrive il risultato direttamente nella globale `MIO_IP` invece di ritornarlo.

funzioni di libreria usate: `ioctl()` con il comando `SIOCGIFADDR` è il cuore: chiede al kernel l'indirizzo dell'interfaccia. `inet_ntop()` converte l'IP dal formato binario a stringa leggibile.

`close()` chiude il socket temporaneo. `strncpy()` copia il nome dell'interfaccia nella struttura di richiesta, `perror()` stampa eventuali errori

2. `free_table()`

Svuota completamente la tabella hash dei protocolli (`table`), liberando la memoria di ogni entry. Viene chiamata **alla fine di ogni finestra di un secondo** (dentro `print_stats`) per azzerare le statistiche e ripartire, e anche in `sigproc` durante la chiusura.

Funzioni di libreria usate: `HASH_ITER` per scorrere le entry in modo sicuro durante la cancellazione (per questo serve la variabile `tmp`, che conserva il puntatore al prossimo elemento prima di distruggere quello corrente), e `HASH_DEL` per staccare l'entry dalla tabella. Poi `free()` della libreria standard per rilasciare la memoria.

3. `free_flow_table()`

Analoga alla precedente, ma per la tabella dei **flussi** (`flow_table`). Viene chiamata alla terminazione del programma (in `sigproc`, e alla fine del `main`/dopo `run_offline`), non a

ogni secondo perché i flussi devono sopravvivere tra una finestra e l'altra per permettere a nDPI di completare la detection. **Vengono utilizzate le stesse funzioni di libreria presenti in `free_table()`.**

4. `init_ndpi()`

Crea e prepara il motore di Deep Packet Inspection di nDPI. Viene chiamata **una volta all'avvio**, nel `main`, prima di processare qualsiasi pacchetto. Internamente passa `NULL` a `ndpi_init_detection_module`, il che significa "nessun contesto globale condiviso".

5. `drop_privileges(const char username)`

Abbassa i privilegi del processo da root all'utente indicato (qui "nobody"). Viene chiamata nel `main` subito dopo `pcap_open_live`: l'apertura della cattura richiede root, ma una volta aperto il socket i privilegi elevati non servono più, e mantenerli sarebbe un rischio di sicurezza. Il problema di questa funzione è che se il rilascio dei privilegi fallisse il programma proseguirebbe da root senza accorgersene.

6. `sigproc(int sig)`

Entra in gioco in modo asincrono: quando l'utente preme Ctrl+C durante la cattura live, il kernel recapita al processo il segnale SIGINT e, essendo stato registrato come gestore tramite `signal()` nel `main`, viene eseguita questa funzione al posto della terminazione predefinita. La funzione interrompe la cattura con `pcap_breakloop`, stampa il riepilogo finale della sessione tramite `print_final_stats`, libera tutte le strutture dati allocate (tabella dei flussi, tabella cumulativa, tabella dell'intervallo, motore nDPI), stampa la durata complessiva della cattura e termina direttamente il programma con `exit(0)`, senza mai restituire il controllo al ciclo di cattura

7. `print_final_stats()`

Stampa il riepilogo conclusivo della sessione di cattura: il numero totale di pacchetti analizzati (`total_pkts_global`) e, per ogni protocollo presente nella tabella cumulativa `total_table`, il conteggio complessivo di pacchetti e byte. Viene chiamata alla terminazione del programma, sia in `sigproc` (chiusura con Ctrl+C in modalità live) sia alla fine di `run_offline` (esaurimento del file pcap), ed è l'unica funzione di stampa che legge la tabella cumulativa anziché quella di intervallo.

8. `free_total_table()`

Analogamente a `free_table` e `free_flow_table`, svuota la tabella hash cumulativa `total_table` liberando la memoria di ogni entry. Viene chiamata solo alla terminazione del programma, necessariamente **dopo** `print_final_stats`, poiché il riepilogo finale legge proprio i dati che questa funzione distrugge

9. `info_argomenti_linea_comando`

Stampa su stderr le istruzioni d'uso del programma: la sintassi delle due modalità di esecuzione e la descrizione delle opzioni disponibili (`-i`, `-f`, `-a`, `-h`). Riceve come parametro `argv[0]`, ossia il nome con cui l'eseguibile è stato invocato, per mostrarlo negli esempi di utilizzo. Viene chiamata dal `main` in tre casi: quando l'utente richiede

esplicitamente l'aiuto con `-h` , quando passa un'opzione non riconosciuta e quando la combinazione di opzioni non è valida (nessuna modalità o entrambe specificate).

10. **update_ip(const char ip_dst, unsigned long bytes, const char ip_srg, const char app_protocol)**

dato un pacchetto già analizzato, aggiorna i contatori di byte per il protocollo corrispondente e classifica il traffico come IN, OUT o TRANSIT. Viene chiamata da `packet_handler` **per ogni pacchetto**, come ultimo passo dopo aver identificato il protocollo. Gli vengono passati quattro parametri: `ip_dst` e `ip_srg` (gli indirizzi sorgente e destinazione in formato stringa, per il confronto con `MI0_IP`), `bytes` (la lunghezza del pacchetto, da accumulare) e `app_protocol` (il nome del protocollo identificato, che funge da chiave nella tabella).

La funzione aggiorna **due tabelle hash distinte**, entrambe di tipo `ip_stat` e indicizzate per nome di protocollo:

- **table (tabella dell'intervallo)**: contiene le statistiche della finestra corrente. Per ogni pacchetto incrementa i campi `bytes` e `pkts` dell'entry del protocollo. Viene azzerata da `print_stats` alla fine di ogni finestra, quindi i suoi valori rappresentano sempre il traffico dell'ultimo secondo.
- **total_table (tabella cumulativa)**: contiene le stesse informazioni (byte e pacchetti per protocollo) ma **non viene mai azzerata** durante l'esecuzione: accumula i totali dell'intera sessione di cattura ed è la sorgente dati del riepilogo finale stampato da `print_final_stats` alla terminazione.

Restituisce `void` : aggiorna le globali `table` , `byte_global` e `byte_local` . Il problema principale è che: La classificazione IN/OUT/TRANSIT è memorizzata **per protocollo, non per flusso**: se lo stesso protocollo (es. TLS) appare sia in traffico locale sia in transito nello stesso secondo, l'ultimo pacchetto sovrascrive il campo `ruolo` del precedente, perdendo informazione.

11. **packet_handler(u_char args, const struct pcap_pkthdr header, const u_char packet)**

la funzione di callback che elabora ogni singolo pacchetto. Esegue il parsing degli header, ricostruisce il flusso di appartenenza, invoca nDPI per la classificazione e infine passa i dati a `update_ip` . Entra in gioco **una volta per ogni pacchetto catturato, ma non è il codice a chiamarla**: è `libpcap` (tramite `pcap_dispatch` in live o esplicitamente in `run_offline`). Gli vengono passati `args` è un puntatore a dati utente personalizzati (io passo NULL perché non mi serve, ma il parametro deve esserci); `header` contiene i metadati del pacchetto (timestamp, lunghezza catturata `caplen` , lunghezza reale `len`); `packet` è il puntatore ai byte grezzi del pacchetto.

funzioni di libreria usate: Di conversione: `inet_ntop` (IP binario→stringa), `ntohl` e `ntohs` (conversione big-endian→little-endian per IP e porte). Di nDPI:

`ndpi_detection_get_sizeof_ndpi_flow_struct` (per sapere quanto allocare),
`ndpi_detection_process_packet` (l'analisi vera e propria), `ndpi_detection_giveup` (stima forzata dopo 80 pacchetti per TCP e 24 per UDP come nDPIreader),
`ndpi_get_proto_name` (ID→nome). Le macro `uthash HASH_FIND` e `HASH_ADD` (qui nelle

varianti per chiave **binaria**, la `FlowKey`, non stringa). Della standard: `calloc`, `memset`, `strlen`.

Questa funzione non funzionerebbe su collegamenti non Ethernet e su pacchetti diversi dal formato IPv4.

12. **run_offline(const char filename)**

Gestisce l'intera modalità **offline**: apre un file `.pcap`, lo scorre pacchetto per pacchetto **rispettando i tempi originali** tra i pacchetti (così da simulare una cattura live), e produce le statistiche a finestre di un secondo. Viene chiamata dal `main` quando l'utente sceglie la modalità "offline", e una volta terminata il programma esce. Per indicare il nome del file che si vuole analizzare l'utente deve inserire come secondo argomento della linea di comando, subito dopo il nome dell'eseguibile, `-f "nomefile.pcap"`. Come nuova funzione esterna viene utilizzata `usleep`, che permette di mettere il programma in pausa per simulare il comportamento live del programma. Nel `main` viene anche effettuato un controllo per verificare se l'utente ha inserito come 3 argomento della funzione, dalla linea di comando, l'IP dell'interfaccia su cui è stata effettuata la cattura che si andrà a simulare. In questo modo si potrà avere un'informazione in più riguardo a se il traffico è IN/OUT/TRANSIT, se l'IP non ci dovesse essere il traffico sarà classificato tutto TRANSIT.

13. **MAIN**

Procede in tre fasi:

Fase 1 — Inizializzazione comune. Il `main` analizza gli argomenti della riga di comando tramite `getopt` (opzioni `-i` per la cattura live, `-f` per l'analisi offline, `-a` per l'IP locale opzionale, `-h` per l'aiuto), verificando che l'utente abbia specificato esattamente una delle due modalità e che l'eventuale IP fornito con `-a` sia sintatticamente valido (controllo effettuato con `inet_pton`); in caso di errore stampa le istruzioni d'uso ed esce. Superata la validazione, registra il gestore del segnale `Ctrl+C`, memorizza l'istante di avvio e inizializza il motore nDPI. Questa parte viene eseguita in entrambe le modalità.

Fase 2 — Biforcazione tra le due modalità. Il programma sceglie il ramo da eseguire in base all'opzione fornita: se l'utente ha specificato `-f`, entra nel ramo offline; se ha specificato `-i`, in quello live.

- **Offline:** imposta `MIO_IP` con l'indirizzo eventualmente fornito tramite `-a` (in sua assenza usa "0.0.0.0", con la conseguenza che tutto il traffico verrà classificato come TRANSIT), quindi delega l'intera analisi a `run_offline`, libera le risorse ed esce con `return 0`. Il ramo offline termina qui e non tocca mai il codice della cattura live.

- **Live:** recupera l'IP locale dell'interfaccia con `get_local_ip`, apre la cattura in tempo reale con `pcap_open_live`, compila e installa il filtro BPF "ip" nel kernel e infine abbassa i privilegi da root all'utente `nobody`.

Fase 3 — Ciclo di cattura live. Entra in un ciclo infinito `while(true)`. Al suo interno, un `do-while` svuota ripetutamente il buffer di `pcap` (chiamando `packet_handler` su ogni pacchetto) finché non è trascorso circa un secondo, misurato sull'orologio reale. Trascorso il secondo, calcola la durata effettiva della finestra, aggiorna il tempo corrente e stampa le

statistiche. Il ciclo si ripete all'infinito: l'unico modo per uscirne è Ctrl+C, che attiva `sigproc`.

Funzioni esterne utilizzate:

- `pcap_open_live` — apre la cattura in tempo reale sull'interfaccia
- `pcap_compile` — traduce la stringa di filtro "ip" in bytecode BPF
- `pcap_setfilter` — installa il filtro compilato nel kernel
- `pcap_dispatch` — processa i pacchetti presenti nel buffer, invocando `packet_handler` per ciascuno
- `pcap_close` — chiude la sessione di cattura (nella parte finale, di fatto irraggiungibile in live)
- `ndpi_exit_detection_module` — spegne il motore nDPI liberando le sue strutture interne

Prerequisiti ed istruzione per esecuzione

1. Installare gli strumenti di compilazione e la libreria libpcap da linea di comando:

```
sudo apt install build-essential autoconf automake libtool pkg-config libpcap-dev
```
 2. Scaricare nDPI (da <https://github.com/ntop/nDPI>) e posizionarlo in `~/nDPI-dev`. Non è necessario compilarlo né installarlo manualmente: alla prima compilazione il Makefile lo compila automaticamente.
Nota: la libreria statica `libndpi.a` viene incorporata nell'eseguibile, così come il file `uthash.h`, prelevato direttamente dal repo.
- Nella cartella del progetto (contenente `monitor_appProto.c` e `Makefile`) compilare con:
`make`. Il Makefile assume che il repository nDPI si trovi in `~/nDPI-dev`, ossia `/home/<utente>/nDPI-dev`. Se il repository si trova in un percorso diverso, è possibile sovrascrivere la variabile `NDPI_ROOT` del Makefile specificandola sulla riga di comando. Ciò avviene aggiungendo a `make` **`NDPI_ROOT=/percorso/di/nDPI-dev`**. Per esempio se si trovasse nella cartella Documents la compilazione del programma sarebbe: **`make NDPI_ROOT=/home/nicola/Documents/nDPI-dev`**.
Successivamente viene prodotto l'eseguibile `monitor_appProto`.
Mentre per ripulire i file generati: `make clean`.
 - Esecuzione:
 - **Cattura live** (richiede privilegi di root):

```
sudo ./monitor_appProto -i <interfaccia> (es. sudo ./monitor_appProto -i eth0)
```
 - **Analisi offline** di un file pcap:

```
./monitor_appProto -f <file.pcap> [-a <indirizzo_ip>]
```

dove `-a` è opzionale e indica l'IP dell'interfaccia su cui è stata effettuata la cattura, per classificare il traffico come IN/OUT; in sua assenza tutto il traffico è etichettato TRANSIT.

- `./monitor_appProto -h` mostra la sintassi completa.
- Per interrompere la cattura live premere **Ctrl+C**: il programma stampa il riepilogo finale della sessione e termina. In modalità offline il programma termina da solo all'esaurimento del file.

Test

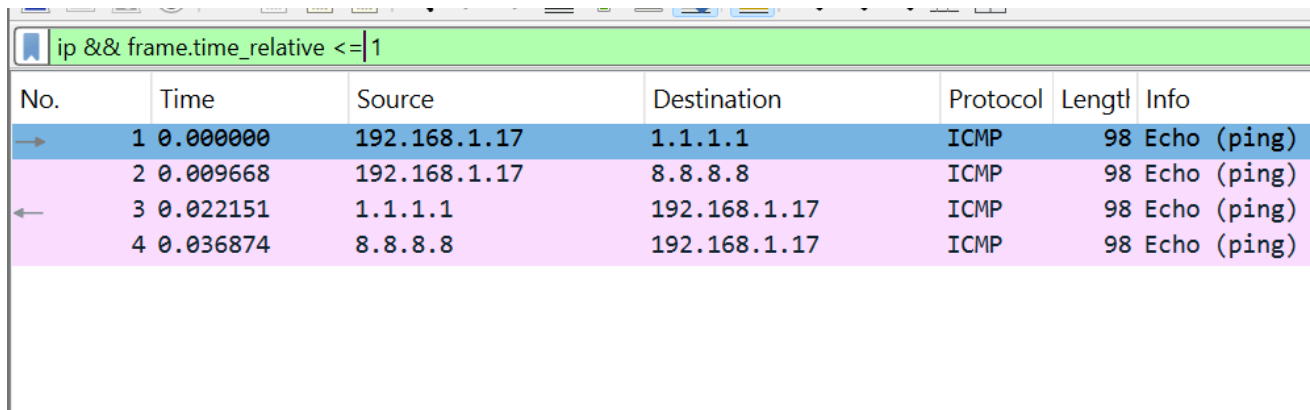
Procedo nel confrontare il contenuto del file *sessione2.pcap* tra il software **wireshark**, visto a lezione, e il programma **monitor_appProto.c** eseguito in modalità offline sul file. Come prima cosa verifico se l'uso di nDPI è corretto confrontando i protocolli applicativi che nDPI assegna ai flussi con quelli mostrati da Wireshark:

NOTA: la numerazione di Wireshark (*frame.time_relative*, che parte da 0) è sfasata di circa 1 rispetto al contatore *[t]* del programma, che parte dal timestamp del primo pacchetto. Quindi l'intervallo 11 di Wireshark corrisponde a *[t=10]* del programma.

Wireshark

- **1 intervallo**

Applicando lo stesso filtro BPF usato dal mio programma, con l'aggiunto della funzione **frame.time_relative <= 1** (seleziona i pacchetti che sono arrivati nel 1 secondo) notiamo che:



The screenshot shows the Wireshark interface with a filter bar at the top containing the BPF filter `ip && frame.time_relative <= 1`. Below the filter, a table displays the first four captured packets, all of which are ICMP Echo (ping) requests.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping)
2	0.009668	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping)
3	0.022151	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping)
4	0.036874	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping)

Il primo pacchetto ricevuto deriva dal ping fatto all'indirizzo IP 1.1.1.1 ed ha protocollo applicativo uguale a ICMP.

Mentre l'ultimo pacchetto del primo secondo di cattura è il pacchetto numero 4.

Perciò se noi osserviamo questo primo intervallo di un secondo possiamo notare come l'unico protocollo presente è quello ICMP, che sommando tutti i byte dei 4 pacchetti dell'intervallo arriviamo a 392 byte.

- **2-3 intervallo**

→	5	1.001225	192.168.1.17	3gpp	ICMP	98	Echo (ping) request
	6	1.010812	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request
←	7	1.024715	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply
	8	1.037946	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply
	9	2.002736	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) request
	10	2.011643	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request
	11	2.026395	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply
	12	2.048069	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply

uguale al primo

- **4 intervallo**

13	3.003642	192.168.1.17	1.1.1.1	ICMP	98	Ec
14	3.013718	192.168.1.17	8.8.8.8	ICMP	98	Ec
15	3.025954	1.1.1.1	192.168.1.17	ICMP	98	Ec
16	3.041215	8.8.8.8	192.168.1.17	ICMP	98	Ec
17	3.707203	192.168.1.18	224.0.0.251	MDNS	401	St
18	3.707204	192.168.1.18	224.0.0.251	MDNS	464	St
19	3.707204	192.168.1.18	224.0.0.251	MDNS	256	St

in questo intervallo notiamo che entra in gioco un nuovo protocollo: MDNS. Notiamo che la somma dei byte dei pacchetti con protocollo MDNS è di 1121 Byte e nessuno di loro ha come indirizzo IP (src o dst) uguale a quello della mia interfaccia. Questo perchè il traffico MDNS è un traffico multicast generato da dispositivi connessi alla rete e viene percepito da chiunque altro sia connesso alla rete locale.

- **5-6 intervallo**

il 5 intervallo è identico al 4 e il 6 è indicato al primo:

ip && frame.time_relative>4 && frame.time_relative < 6						
o.	Time	Source	Destination	Protocol	Length	Ir
▶	20	4.004713	192.168.1.17	1.1.1.1	ICMP	98
	21	4.014670	192.168.1.17	8.8.8.8	ICMP	98
-	22	4.029781	1.1.1.1	192.168.1.17	ICMP	98
	23	4.043121	8.8.8.8	192.168.1.17	ICMP	98
	25	4.729751	192.168.1.18	224.0.0.251	MDNS	401
	26	4.731155	192.168.1.18	224.0.0.251	MDNS	464
	27	4.731155	192.168.1.18	224.0.0.251	MDNS	256
	28	5.005731	192.168.1.17	1.1.1.1	ICMP	98
	29	5.015583	192.168.1.17	8.8.8.8	ICMP	98
	30	5.033101	1.1.1.1	192.168.1.17	ICMP	98
	31	5.053435	8.8.8.8	192.168.1.17	ICMP	98

- **7 intervallo**

ip && frame.time_relative>6 && frame.time_relative < 7						
o.	Time	Source	Destination	Protocol	Length	Info
32	6.007674	192.168.1.17	1.1.1.1	ICMP	98	Echo
35	6.017027	192.168.1.1	192.168.1.17	NBNS	92	Name
36	6.017051	192.168.1.17	192.168.1.1	ICMP	120	Desti
37	6.017942	192.168.1.17	8.8.8.8	ICMP	98	Echo
38	6.030210	1.1.1.1	192.168.1.17	ICMP	98	Echo
39	6.049088	8.8.8.8	192.168.1.17	ICMP	98	Echo

Nel 7 intervallo troviamo un nuovo tipo di protocollo, ovvero il protocollo NBNS (92 byte) che nDPI chiama NetBIOS. Che verrà etichettato come traffico di ingresso (IN) visto che ha come IP dst l'indirizzo IP della mia interfaccia. Collegato a questo c'è la particolarità, rispetto agli intervalli precedenti di avere un singolo pacchetto ICMP, più grande rispetto agli altri, in più. Ciò è dovuto al fatto che si tratta di un messaggio ICMP di tipo 'Destination Unreachable (Port unreachable)' e non di un normale echo request/reply. Questo tipo di ICMP è più grande perché, secondo lo standard, incorpora l'header IP e i primi byte del pacchetto che ha generato l'errore. Nello specifico, è la risposta del sistema alla query NetBIOS in ingresso: non essendoci un servizio in ascolto sulla porta richiesta, la macchina ha notificato al mittente l'irraggiungibilità della porta."

- **8 - 9- 10 intervallo**

ip && frame.time_relative> 7 && frame.time_relative < 10						
o.	Time	Source	Destination	Protocol	Length	Info
40	7.009833	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping)
41	7.017673	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping)
42	7.035201	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping)
43	7.045224	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping)
44	8.010723	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping)
45	8.018664	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping)
46	8.045054	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping)
47	8.046108	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping)
48	9.011795	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping)
49	9.019651	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping)
50	9.031420	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping)
51	9.047356	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping)

Uguali al primo intervallo data la presenza dei pacchetti relativi ai due ping in corso.

- **11-12 intervallo**

Ho voluto inserire insieme questi due intervalli perchè si osserva una conversazione applicativa completa verso un server HTTPS. Nell'intervallo 10 avviene la fase di setup: risoluzione DNS di example.com, handshake TCP a tre vie sulla porta 443 e avvio dell'handshake TLS con Client Hello (SNI=example.com), il campo che permette a nDPI di classificare il flusso come TLS. Nell'intervallo 11 prosegue lo stesso flusso con il trasferimento dei dati applicativi cifrati (Application Data).

ip && frame.time_relative > 10 && frame.time_relative < 11						
	Time	Source	Destination	Protocol	Length	Info
52	10.012759	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) request id=0x4d7f, seq=
53	10.020693	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request id=0x4d7e, seq=
54	10.034926	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply id=0x4d7f, seq=
55	10.054262	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply id=0x4d7e, seq=
58	10.766984	192.168.1.17	192.168.1.1	DNS	82	Standard query 0xd8a9 A example.com
59	10.768897	192.168.1.17	192.168.1.1	DNS	82	Standard query 0xe5a3 AAAA example.
60	10.793412	192.168.1.1	192.168.1.17	DNS	114	Standard query response 0xd8a9 A ex
61	10.796240	192.168.1.1	192.168.1.17	DNS	138	Standard query response 0xe5a3 AAAA
62	10.797049	192.168.1.17	104.20.23.154	TCP	74	49154 → 443 [SYN] Seq=0 Win=64240 L
63	10.819855	104.20.23.154	192.168.1.17	TCP	74	443 → 49154 [SYN, ACK] Seq=0 Ack=1
64	10.819903	192.168.1.17	104.20.23.154	TCP	66	49154 → 443 [ACK] Seq=1 Ack=1 Win=6
65	10.825409	192.168.1.17	104.20.23.154	TLSv1.2	583	Client Hello (SNI=example.com)
66	10.849706	104.20.23.154	192.168.1.17	TCP	66	443 → 49154 [ACK] Seq=1 Ack=518 Win=
67	10.868530	104.20.23.154	192.168.1.17	TLSv1.2	1178	[TCP Previous segment not captured]
68	10.868552	192.168.1.17	104.20.23.154	TCP	78	[TCP Dup ACK 64#1] 49154 → 443 [ACK

ip && frame.time_relative > 11 && frame.time_relative < 12						
	Time	Source	Destination	Protocol	Length	Info
69	11.013754	192.168.1.17	1.1.1.1	ICMP	98	Echo
70	11.022748	192.168.1.17	8.8.8.8	ICMP	98	Echo
71	11.032268	1.1.1.1	192.168.1.17	ICMP	98	Echo
72	11.049149	8.8.8.8	192.168.1.17	ICMP	98	Echo
73	11.123734	104.20.23.154	192.168.1.17	TCP	1506	[TCP
74	11.123768	192.168.1.17	104.20.23.154	TCP	78	4915
75	11.145000	104.20.23.154	192.168.1.17	TLSv1.2	1506	[TCP
76	11.145070	192.168.1.17	104.20.23.154	TCP	66	4915
77	11.156302	192.168.1.17	104.20.23.154	TLSv1.2	146	Char
78	11.159959	192.168.1.17	104.20.23.154	TLSv1.2	152	Appl
79	11.161058	192.168.1.17	104.20.23.154	TLSv1.2	125	Appl
80	11.177160	104.20.23.154	192.168.1.17	TLSv1.2	610	Appl
81	11.177463	192.168.1.17	104.20.23.154	TLSv1.2	97	Appl
82	11.179278	104.20.23.154	192.168.1.17	TLSv1.2	97	Appl
83	11.179278	104.20.23.154	192.168.1.17	TCP	66	443
84	11.185168	104.20.23.154	192.168.1.17	TLSv1.2	226	Appl
85	11.185168	104.20.23.154	192.168.1.17	TLSv1.2	656	Appl
86	11.185168	104.20.23.154	192.168.1.17	TLSv1.2	97	Appl
87	11.185397	192.168.1.17	104.20.23.154	TCP	66	4915
88	11.189134	192.168.1.17	104.20.23.154	TLSv1.2	90	Appl
89	11.192837	192.168.1.17	104.20.23.154	TCP	66	4915
90	11.209252	104.20.23.154	192.168.1.17	TCP	66	443
91	11.209253	104.20.23.154	192.168.1.17	TCP	66	443
92	11.209288	192.168.1.17	104.20.23.154	TCP	66	4915
93	11.213200	104.20.23.154	192.168.1.17	TCP	66	443

La prima foto corrisponde all'intervallo 11 e la seconda all'intervallo 12. Nell'intervallo 11 si hanno 4 pacchetti ICMP dovuti ai ping (392 Byte), 4 pacchetti DNS (416 Byte), 3 pacchetti TCP che corrispondono all'handshake (214 Byte) e 2 pacchetti TLS (1761 Byte) mischiati ad altri 2 pacchetti TCP (124Byte).

Invece la seconda foto corrisponde all'intervallo 12 in cui si hanno lo stesso numero di pacchetti ICP derivanti dagli stessi ping e un insieme di 11 pacchetti TLS (4781 Byte) combinati a 10 pacchetti TCP (2112 Byte).

- Intervallo 16-17

ip && frame.time_relative > 15 && frame.time_relative < 16						
	Time	Source	Destination	Protocol	Length	Info
106	15.019599	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) request id=0x4d7f,
107	15.026544	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request id=0x4d7e,
108	15.041302	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply id=0x4d7f,
109	15.053214	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply id=0x4d7e,
110	15.935314	192.168.1.17	104.20.23.154	TCP	74	49160 → 443 [SYN] Seq=0 Win=6424
111	15.957717	104.20.23.154	192.168.1.17	TCP	74	443 → 49160 [SYN, ACK] Seq=0 Ack
112	15.957759	192.168.1.17	104.20.23.154	TCP	66	49160 → 443 [ACK] Seq=1 Ack=1 Wi
113	15.962696	192.168.1.17	104.20.23.154	TLSv1.2	583	Client Hello (SNI=example.com)
114	15.988723	104.20.23.154	192.168.1.17	TCP	66	443 → 49160 [ACK] Seq=1 Ack=518
ip && frame.time_relative > 16 && frame.time_relative < 17						
	Time	Source	Destination	Protocol	Length	Info
115	16.004989	104.20.23.154	192.168.1.17	TLSv1.2	1178	[TCP Previous segment not
116	16.005009	192.168.1.17	104.20.23.154	TCP	78	[TCP Dup ACK 112#1] 49160
117	16.022623	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) request id=0x
118	16.027518	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request id=0x
119	16.042015	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply id=0x
120	16.054119	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply id=0x
121	16.266386	104.20.23.154	192.168.1.17	TCP	1506	[TCP Retransmission] 443 →
122	16.266417	192.168.1.17	104.20.23.154	TCP	78	49160 → 443 [ACK] Seq=518
123	16.287084	104.20.23.154	192.168.1.17	TLSv1.2	1506	[TCP Out-Of-Order] , Ignor
124	16.287117	192.168.1.17	104.20.23.154	TCP	66	49160 → 443 [ACK] Seq=518
125	16.292345	192.168.1.17	104.20.23.154	TLSv1.2	146	Change Cipher Spec, Applic
126	16.293281	192.168.1.17	104.20.23.154	TLSv1.2	152	Application Data
127	16.294157	192.168.1.17	104.20.23.154	TLSv1.2	125	Application Data
128	16.315295	104.20.23.154	192.168.1.17	TLSv1.2	619	Application Data, Applicat
129	16.315296	104.20.23.154	192.168.1.17	TCP	66	443 → 49160 [ACK] Seq=3993
130	16.317486	192.168.1.17	104.20.23.154	TLSv1.2	97	Application Data
131	16.320368	104.20.23.154	192.168.1.17	TLSv1.2	225	Application Data
132	16.320369	104.20.23.154	192.168.1.17	TLSv1.2	656	Application Data
133	16.320369	104.20.23.154	192.168.1.17	TLSv1.2	97	Application Data
134	16.330642	192.168.1.17	104.20.23.154	TLSv1.2	90	Application Data
135	16.335895	192.168.1.17	104.20.23.154	TCP	66	49160 → 443 [FIN, ACK] Seq
136	16.351196	104.20.23.154	192.168.1.17	TCP	66	443 → 49160 [ACK] Seq=5325
137	16.352595	104.20.23.154	192.168.1.17	TCP	66	443 → 49160 [FIN, ACK] Seq
138	16.352617	192.168.1.17	104.20.23.154	TCP	66	49160 → 443 [ACK] Seq=799
139	16.356317	104.20.23.154	192.168.1.17	TCP	66	443 → 49160 [ACK] Seq=5327

Nella prima immagine troviamo l'intervallo 16, in cui sono presenti i soliti 4 pacchetti dovuti ai ping con protocollo ICMP (392 byte). In più, come già accaduto nell'intervallo 11, troviamo i tre pacchetti TCP che formano l'handshake con il server di `www.example.com` (avviato dal comando `curl` eseguito su un altro terminale) e, successivamente, il Client Hello TLS che dà inizio alla conversazione cifrata. Ho voluto accostare l'intervallo 16 al 17 perché, come nel caso degli intervalli 11 e 12, la conversazione iniziata con il server di `example.com` prosegue nell'intervallo successivo con l'invio e la ricezione di ulteriori pacchetti TLS. La particolarità è che questa volta non compaiono pacchetti DNS.

C'è però un errore perché guardando il tempo di arrivo tra il primo pacchetto dell'intervallo 16 e il tempo di arrivo del secondo pacchetto del 17 possiamo notare come essi siano parte dello stesso intervallo. Perciò senza contare quei due il numero di byte corrispondenti ai pacchetti TLS dell'intervallo 17 è pari a 5759 Byte.

- **Intervallo 13-14-15 e 18-19**

ip && frame.time_relative>12 && frame.time_relative < 15

	Time	Source	Destination	Protocol	Length	Info
94	12.015652	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) r
95	12.023591	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) r
96	12.045982	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) r
97	12.050392	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) r
98	13.016949	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) r
99	13.024758	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) r
100	13.039306	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) r
101	13.053030	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) r
102	14.018921	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) r
103	14.025633	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) r
104	14.041858	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) r
105	14.052718	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) r

ip && frame.time_relative>17 && frame.time_relative < 19

	Time	Source	Destination	Protocol	Length	Info
140	17.023762	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) request
141	17.029679	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request
142	17.045026	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply
143	17.057375	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply
144	18.024695	192.168.1.17	1.1.1.1	ICMP	98	Echo (ping) request
145	18.030681	192.168.1.17	8.8.8.8	ICMP	98	Echo (ping) request
146	18.045243	1.1.1.1	192.168.1.17	ICMP	98	Echo (ping) reply
147	18.068842	8.8.8.8	192.168.1.17	ICMP	98	Echo (ping) reply

Intervalli uguali al primo, in cui sono presenti solo i pacchetti derivanti dai due ping. La prima foto corrisponde ai primi tre intervalli mentre la seconda agli ultimi due.

Monitor_appProto.c

il programma è stato avviato con questo comando:

```
nicobrillo@nicola:~/progetto$ sudo ./monitor_appProto -f sessione2.pcap -a 192.168.1.17
```

- **1 intervallo**

```
[t = 0]-----
Throughput globale: 391.52 B/s
Throughput locale: 391.52 B/s
Protocolli osservati:
ICMP                      392 Byte      4 pkt  [OUT]
```

Possiamo facilmente notare come nel primo intervallo di tempo del mio programma (1 secondo) l'unico protocollo applicativo osservato è quello ICMP, il numero totale di byte osservati è circa 392 byte e sono tutti derivanti da pacchetti di conversazioni avviate dalla mia interfaccia di rete (ping).

- **2-3 intervallo**

```
[t = 1]-----
Throughput globale: 391.41 B/s
Throughput locale: 391.41 B/s
Protocolli osservati:
  ICMP                      392 Byte  [OUT]

[t = 2]-----
Throughput globale: 391.65 B/s
Throughput locale: 391.65 B/s
Protocolli osservati:
  ICMP                      392 Byte  [OUT]
```

uguale al primo

- **4 intervallo**

```
[t = 3]-----
Throughput globale: 1511.38 B/s
Throughput locale: 391.58 B/s
Protocolli osservati:
  ICMP                      392 Byte  4 pkt  [OUT]
  MDNS                      1121 Byte  3 pkt  [TRANSIT]
```

Come vediamo il programma rispetta ciò che viene osservato da wireshark, indicando come unici due protocolli individuati ICMP e MDNS entrambi con circa il giusto numero di Byte.

Ovviamente i pacchetti MDNS sono stati etichettati come TRANSIT non avendo sia nell'ip srg e nell'ip dst l'indirizzo ip della mia interfaccia.

- **5-6 intervallo**

il 5 intervallo è identico al 4 e il 6 identico al primo:

```
[t = 4]-----
Throughput globale: 1511.46 B/s
Throughput locale: 391.60 B/s
Protocolli osservati:
  ICMP                      392 Byte  4 pkt  [OUT]
  MDNS                      1121 Byte  3 pkt  [TRANSIT]

[t = 5]-----
Throughput globale: 391.24 B/s
Throughput locale: 391.24 B/s
Protocolli osservati:
  ICMP                      392 Byte  4 pkt  [OUT]
```

- **7 intervallo**

```
[t = 6]-----
Throughput globale: 602.70 B/s
Throughput locale: 602.70 B/s
Protocolli osservati:
  ICMP                      512 Byte  5 pkt  [OUT]
  NetBIOS                   92 Byte   1 pkt  [IN]
```

Il numero di byte per i vari protocolli è corretto rispetto a ciò che indica wireshark, come sono corrette le etichette IN e OUT per NetBIOS, perchè è una query che proviene dall'esterno, e per ICMP.

- **8-9-10 intervallo**

```
[t = 7]-----
Throughput globale: 391.65 B/s
Throughput locale: 391.65 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]

[t = 8]-----
Throughput globale: 391.58 B/s
Throughput locale: 391.58 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]

[t = 9]-----
Throughput globale: 391.62 B/s
Throughput locale: 391.62 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]
```

uguale all'intervallo 1 dato che sono presenti solo i pacchetti dei due ping.

- **11-12 intervallo**

```
[t = 10]-----
Throughput globale: 2924.09 B/s
Throughput locale: 2924.09 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]
  DNS                 416 Byte      4 pkt  [OUT]
  Unknown             214 Byte      3 pkt  [OUT]
  TLS                 1905 Byte     4 pkt  [OUT]

[t = 11]-----
Throughput globale: 6294.05 B/s
Throughput locale: 6294.05 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]
  TLS                 5914 Byte     21 pkt  [IN]
```

Come possiamo notare, nell'intervallo 10 sono presenti 4 protocolli diversi, tra cui Unknown. I protocolli in questione derivano: dai soliti 4 pacchetti ICMP generati dai ping; da 4 pacchetti DNS generati dal comando `curl https://www.example.com`, che servono a risolvere il nome di dominio `example.com` nel suo indirizzo IP prima di poter stabilire la connessione; e infine da pacchetti per i quali nDPI non è ancora riuscito a rilevare il protocollo (etichettati come Unknown), pur correttamente classificati come traffico in uscita (OUT). Questi ultimi corrispondono ai pacchetti iniziali dell'handshake TCP tra la mia interfaccia di rete e il server di `www.example.com`: non vengono riconosciuti perché contengono solo informazioni di livello trasporto (SYN, SYN-ACK, ACK) e nessun dato applicativo utile a nDPI per identificare il protocollo. Infatti, quando arriva il pacchetto con il Client Hello TLS (pacchetto 65), nDPI riconosce il flusso come TLS e, da quel momento, tutti i pacchetti successivi dello stesso flusso vengono classificati come TLS. Il numero di byte per ogni protocollo è quindi coerente, considerando che i pacchetti TCP/TLS identificati vengono aggregati sotto l'etichetta TLS, mentre i primi pacchetti di solo handshake TCP, non ancora classificabili, restano conteggiati come Unknown.

Nell'intervallo 12 troviamo sempre i soliti 4 pacchetti ICMP, ma a differenza dell'intervallo precedente i pacchetti TCP vengono fin da subito riconosciuti come facenti parte della conversazione TLS, grazie alla tabella dei flussi che ha mantenuto le informazioni dell'intervallo precedente. L'altra differenza è che il TLS è stato etichettato come IN: ciò avviene perché si tratta della prosecuzione della conversazione iniziata nell'intervallo 11, e il primo pacchetto TLS dell'intervallo 12 è in arrivo dal server esterno (in ingresso verso la mia interfaccia).

- **Intervallo 16-17**

```
[t = 15]-----
Throughput globale: 2503.43 B/s
Throughput locale: 2503.43 B/s
Protocolli osservati:
  ICMP          392 Byte      4 pkt [OUT]
  Unknown       214 Byte      3 pkt [OUT]
  TLS           1905 Byte     4 pkt [OUT]

[t = 16]-----
Throughput globale: 6144.00 B/s
Throughput locale: 6144.00 B/s
Protocolli osservati:
  ICMP          392 Byte      4 pkt [OUT]
  TLS           5759 Byte    19 pkt [IN]
```

In questa accoppiata di intervalli succede esattamente la stessa cosa che è successa negli intervalli 11 e 12, ovvero viene effettuato un curl su un terminale parallelo al solito www.example.com, ciò quindi genera più o meno lo stesso numero di byte per gli stessi protocolli con, però, una differenza importante : non compaiono pacchetti DNS all'inizio della conversazione . Ciò è dovuto dal fatto che i pacchetti DNS servono inizialmente per risolvere l'indirizzo IP di www.example.com solo che avendolo già trovato negli intervalli 11-12 esso viene salvato nella cache DNS del sistema operativo per la durata del TTL del record, permettendo di recuperarlo senza nuove query finché la cache è valida.

- **Intervallo 13-14-15 e 18-19**

Intervalli uguali al primo, in cui sono presenti solo i pacchetti derivanti dai due ping.

```
[t = 12]-----
Throughput globale: 391.49 B/s
Throughput locale: 391.49 B/s
Protocolli osservati:
  ICMP          392 Byte      4 pkt [OUT]

[t = 13]-----
Throughput globale: 391.23 B/s
Throughput locale: 391.23 B/s
Protocolli osservati:
  ICMP          392 Byte      4 pkt [OUT]

[t = 14]-----
Throughput globale: 391.73 B/s
Throughput locale: 391.73 B/s
Protocolli osservati:
  ICMP          392 Byte      4 pkt [OUT]
```

Questi sono i primi 3 intervalli

```

[t = 17]-----
Throughput globale: 391.63 B/s
Throughput locale: 391.63 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]

[t = 18]-----
Throughput globale: 392.00 B/s
Throughput locale: 392.00 B/s
Protocolli osservati:
  ICMP                392 Byte      4 pkt  [OUT]

```

Questi sono gli ultimi 2.

Calcolo Throughput

Il throughput è la quantità di dati che attraversa una determinata rete in un unità di tempo (B/s). Nel mio programma sono calcolati ed espressi due tipi di throughput differente: Il **throughput globale** e il **throughput locale**.

Il **throughput globale** raffigura i Byte che passano ogni secondo sulla rete considerando tutti i pacchetti visti sull'interfaccia precedentemente filtrati dal filtro BPF.

Il **throughput locale** invece rappresenta quanti Byte passano sulla rete ogni secondo considerando i pacchetti che coinvolgono direttamente la mia interfaccia (anch'essi filtrati).

Entrambi sono calcolati dividendo la somma dei byte di tutti i pacchetti, interessati per i vari throughput, per la durata reale della finestra, che spesso è sempre leggermente maggiore di 1 secondo. Perciò dividere per la durata reale (es. 1.002 s) anziché per un valore assunto di 1.0 fornisce la misura più **accurata** del throughput, al costo di una piccola differenza rispetto al conteggio grezzo dei byte.

File di test

File sessione2.pcap (trovatosi nella cartella del progetto) creato attraverso l'uso del comando **tcpdump** sull'interfaccia di rete della mia VM linux oracle. Precisamente il comando usato è stato : **sudo tcpdump -i enp0s3 -w sessione2.pcap**. Con enp0s3 l'interfaccia di rete della mia VM.

Mentre veniva effettuata la cattura dei pacchetti sono stati contemporaneamente attivati due **ping** da due terminali della VM (uno a 1.1.1.1 e un altro a 8.8.8.8), che sono stati tenuti accessi durante tutto il tempo di cattura, e ogni tanto dei **curl** al sito <https://www.example.com> per fare catturare a tcpdump un numero maggiore di pacchetti ai fini di migliorare il testing.

Criticità del programma

- La `flow_table` **cresce all'infinito**. I flussi vengono aggiunti ma **mai rimossi** durante l'esecuzione di una sessione live lunga e ciò può portare a un degradamento notevole delle prestazioni

- Per cercare di simulare correttamente il comportamento del mio programma in una sessione di cattura offline ho impostato una **usleep** che rallenta volutamente l'uscita dell'output. Quindi l'analisi di un file di un ora richiede un ora reale anche nella modalità offline. Ciò è eliminabile eliminando la **usleep**.
- Il ruolo IN/OUT/TRANSIT è assegnato per protocollo non per flusso, perciò se nello stesso secondo arrivano due flussi TLS, uno locale e uno di transito, finiscono nella stessa entry TLS a cui è assegnata una delle tre etichette (IN/OUT/TRANSIT) in base a quello che è arrivato prima e i byte si sommano tutti lì.
- **dt** non esattamente di un secondo. Discrepanza tra throughput e conteggio Byte grezzi nella cattura live.
- Il nome globale e locale per i throughput possono essere fuorvianti perchè non includono tutti i pacchetti che passano dall'interfaccia ma per una scelta progettuale solo quelli IPv4 e filtrati con il filtro BPF "ip
- se il rilascio dei privilegi fallisce il programma continua con i privilegi da root